

Lesson 4 – Strings and String Methods

ReDI School · Introduction to Programming

What did we learn last week?

- **Arithmetic operators:** `+` `-` `*` `/` `//` `%` `**`
- Brackets come first in calculations
- `abs()` removes the minus sign · `round()` controls decimal places
- `input()` always returns a string – convert with `int()` or `float()`
- `str()` converts a number to text

Any questions before we start?

Today's Plan

Part	Topic
Part 1	String indexing and slicing
Part 2	Basic string methods
Part 3	Escape characters
	Break
Part 4	Checking and searching strings
Part 5	split() and join()
Part 6	f-strings

Part 1 – String Indexing and Slicing

Strings are sequences

Every character in a string has a **position number** called an index. Counting starts at 0.

```
name = "Python"  
#      012345
```



```
print(name[0])    # P  - first character  
print(name[1])    # y  
print(name[5])    # n  - last character  
  
# Negative index - count from the end  
print(name[-1])   # n  - last character  
print(name[-2])   # o  - second to last  
print(name[-6])   # P  - same as name[0]
```



Python always starts counting from **0**, not 1. The last character is at index **-1**.

String slicing

A **slice** extracts a piece of a string: `name[start:end]`

The `end` index is **not included**.

```
name = "Python"
#      012345

print(name[0:3])  # Pyt  - index 0, 1, 2
print(name[2:5])  # tho  - index 2, 3, 4
print(name[0:6])  # Python - the whole word
```



```
# Leave out start or end to go to the beginning or end
print(name[:3])    # Pyt  - same as name[0:3]
print(name[3:])    # hon  - from index 3 to the end
print(name[:])     # Python - full copy

# Step - every Nth character
print(name[::-2])  # Pto  - every 2nd character
print(name[::-1])  # nohtyP - reversed!
```





Quick Exercise – Indexing and Slicing

```
city = "Amsterdam"
```



1. Print the first letter
2. Print the last letter
3. Print the first 3 letters
4. Print the city name reversed
5. Print every second letter



8 minutes

Quick Exercise – Solution

```
city = "Amsterdam"

print(city[0])      # A
print(city[-1])     # m
print(city[:3])     # Ams
print(city[::-1])   # madretsmA
print(city[::-2])   # Asedm
```



Part 2 – Basic String Methods

Changing case

```
name = "sara ahmed"

print(name.upper())      # SARA AHMED
print(name.lower())      # sara ahmed
print(name.title())      # Sara Ahmed
print(name.capitalize()) # Sara ahmed – only first letter
```



Why .lower() matters for comparisons:

```
answer = input("Do you want to continue? (yes/no) ")

# ❌ Fragile – only works if user types exactly "yes"
if answer == "yes":
    print("Continuing... ")

# ✅ Better – works for "Yes", "YES", "yes", "YeS"
if answer.lower() == "yes":
    print("Continuing... ")
```



Removing spaces – strip()

```
username = "  sara  "

print(username.strip())    # "sara" – removes both sides
print(username.lstrip())  # "sara  " – removes left only
print(username.rstrip())  # "  sara" – removes right only
```



Users often accidentally type extra spaces. Always `.strip()` input before comparing or storing it.

```
email = input("Enter your email: ")
email = email.strip().lower()  # clean it up in one line
print("Saved:", email)
```



Replacing text – replace()

```
sentence = "I love Java"
print(sentence.replace("Java", "Python"))    # I love Python

# Replace all occurrences
text = "cat sat on a cat mat"
print(text.replace("cat", "dog"))            # dog sat on a dog mat

# Replace a character
phone = "012-345-678"
print(phone.replace("-", ""))                # 012345678
```



`.replace(old, new)` returns a new string – it does not change the original. Assign it back if you want to keep the change: `sentence = sentence.replace("Java", "Python")`



Quick Exercise – String Methods

```
user_input = "  HELLO, my name is SARA.  "
```



1. Remove the spaces from both sides
2. Convert everything to lowercase
3. Replace "sara" with your own name
4. Print the length of the cleaned string



8 minutes

Quick Exercise – Solution

```
user_input = "    HELLO, my name is SARA.    "

cleaned = user_input.strip()
print(cleaned)                # HELLO, my name is SARA.

lower    = cleaned.lower()
print(lower)                  # hello, my name is sara.

replaced = lower.replace("sara", "arun")
print(replaced)               # hello, my name is arun.

print(len(replaced))          # 26
```



Part 3 – Escape Characters

Escape characters

Sometimes you need special characters inside a string. Use a backslash `\` to escape them.

```
# \n - new line
print("Line 1\nLine 2\nLine 3")
# Line 1
# Line 2
# Line 3

# \t - tab (indent)
print("Name:\tSara")
print("City:\tBerlin")
# Name:   Sara
# City:   Berlin

# \' and \" - quotes inside strings
print("She said \"Hello!\"")    # She said "Hello!"
print('It\'s a great day')      # It's a great day

# \\ - a literal backslash
print("C:\\Users\\sara")        # C:\Users\sara
```



Using escape characters for formatting

```
name = "Sara"
score = 95
grade = "A"

# Build a formatted report card
print("=== Report Card ===")
print(f"Name:\t{name}")
print(f"Score:\t{score}")
print(f"Grade:\t{grade}")
print("=====")
```



```
=== Report Card ===
Name:   Sara
Score:  95
Grade:  A
=====
```



`\n` and `\t` are the two you will use most often. They make output much easier to read.



Break – 10 minutes

Back in 10 minutes

Part 4 – Checking and Searching Strings

Checking string content

```
age_input = "25"
name_input = "Sara"

# .isdigit() - True if all characters are digits
print(age_input.isdigit())    # True
print(name_input.isdigit())   # False
print("12.5".isdigit())       # False - decimal point is not a digit

# .isalpha() - True if all characters are letters
print(name_input.isalpha())   # True
print(age_input.isalpha())    # False
print("Sara Ahmed".isalpha()) # False - space is not a letter
```



Use `.isdigit()` before converting with `int()` to avoid crashes:

```
value = input("Enter your age: ")
if value.isdigit():
    age = int(value)
    print("Your age is", age)
else:
    print("That is not a valid age!")
```



startswith(), endswith() and in

```
email = "sara@redi.de"
filename = "report.pdf"

# startswith() and endswith()
print(email.endswith(".de"))      # True
print(email.endswith(".com"))     # False
print(filename.endswith(".pdf"))  # True
print(filename.startswith("rep")) # True

# in – check if a string contains something
print("@" in email)               # True
print(".de" in email)             # True
print("gmail" in email)           # False

# Combine them to validate an email
if "@" in email and "." in email:
    print("Looks like a valid email")
```



count() and find()

```
sentence = "the cat sat on the mat"

# count() - how many times does something appear?
print(sentence.count("the"))    # 2
print(sentence.count("at"))     # 3
print(sentence.count("dog"))    # 0

# find() - where does something first appear?
print(sentence.find("cat"))      # 4 - position of "c"
print(sentence.find("mat"))      # 19
print(sentence.find("dog"))      # -1 - not found
```



`find()` returns the index of the first match, or `-1` if it is not found.



Quick Exercise – Checking Strings

1. Ask the user to enter a number. Use `.isdigit()` to check if it really is a number before converting it with `int()`.
2. Ask the user to enter an email address. Check that it contains `"@"` and `"."`. Print `"Valid"` or `"Invalid"`.
3. Given `text = "banana"`, print how many times `"a"` appears.



10 minutes

Quick Exercise – Solution

```
# 1. Safe number input
value = input("Enter a number: ")
if value.isdigit():
    print("The number is:", int(value))
else:
    print("That is not a number!")

# 2. Email check
email = input("Enter your email: ")
if "@" in email and "." in email:
    print("Valid")
else:
    print("Invalid")

# 3. Count letters
text = "banana"
print(text.count("a"))    # 3
```



Part 5 – split() and join()

split() – text into a list

```
sentence = "I love learning Python"

words = sentence.split()      # split on spaces by default
print(words)                  # ['I', 'love', 'learning', 'Python']
print(words[0])                # I
print(words[-1])               # Python
print(len(words))              # 4 – number of words

# Split on a specific character
date = "2026-09-30"
parts = date.split("-")
print(parts)                   # ['2026', '09', '30']
print(parts[0])                 # 2026 – year
print(parts[2])                 # 30 – day

csv_row = "Sara,28,Berlin"
data = csv_row.split(",")
print(data[0])                  # Sara
print(data[1])                  # 28
```



join() – list back into text

```
words = ["I", "love", "Python"]

sentence = " ".join(words)
print(sentence)           # I love Python

# Join with a different separator
print(", ".join(words))   # I, love, Python
print("-".join(words))    # I-love-Python

# Practical: clean up a name
parts = [" sara ", " ahmed "]
clean = " ".join(part.strip().title() for part in parts)
print(clean)              # Sara Ahmed
```



The pattern is: `"separator".join(list)`. Whatever is in quotes goes between each item.



Quick Exercise – split() and join()

```
sentence = "Python is fun to learn"
```



1. Split the sentence into words and print each word on its own line
2. Print how many words are in the sentence
3. Join the words back together with a `-` between each word



8 minutes

Quick Exercise – Solution

```
sentence = "Python is fun to learn"
words    = sentence.split()

# 1. Print each word
for word in words:
    print(word)

# 2. Word count
print(len(words))          # 5

# 3. Join with -
print("-".join(words))     # Python-is-fun-to-learn
```



Part 6 – f-strings

f-strings – a cleaner way to build text

```
name = "Sara"
age = 25
city = "Berlin"

# Old way – messy with +
print("Hello, " + name + "! You are " + str(age) + " years old.")

# f-string – much cleaner
print(f"Hello, {name}! You are {age} years old.")
print(f"{name} lives in {city}.")
```



```
# You can put expressions inside {}
score = 82
total = 100
print(f"Score: {score}/{total} ({score/total*100:.1f}%)")
# Score: 82/100 (82.0%)

a = 5
b = 3
print(f"{a} + {b} = {a + b}")    # 5 + 3 = 8
```



Formatting numbers in f-strings

```
price = 9.5

# .2f – always show 2 decimal places
print(f"Price: €{price:.2f}")      # Price: €9.50

# Useful for money
total    = 1234.5
discount = total * 0.1
final    = total - discount
print(f"Original: €{total:.2f}")   # €1234.50
print(f"Discount: €{discount:.2f}") # €123.45
print(f"Final:     €{final:.2f}")   # €1111.05
```



```
# Multiline f-string output
name  = "Sara"
score = 95
print(f"Student: {name}\nScore:   {score}\nGrade:   A")
# Student: Sara
# Score:   95
# Grade:   A
```





Quick Exercise – f-strings

1. Create variables: `name` , `age` , `city` . Print one sentence using an f-string that includes all three.
2. Ask the user for a price. Print it with and without 21% VAT, both formatted to 2 decimal places.



8 minutes

Quick Exercise – Solution

```
# 1. Profile sentence
name = "Sara"
age = 25
city = "Berlin"
print(f"My name is {name}, I am {age} years old and I live in {city}.")

# 2. VAT calculator
price = float(input("Enter a price (€): "))
total = price * 1.21
print(f"Without VAT: €{price:.2f}")
print(f"With VAT:    €{total:.2f}")
```





Break – 10 minutes

Back in 10 minutes – mini project time!

Common Mistakes

✗ Case-sensitive comparisons

"Python" == "python" is False. Always use `.lower()` before comparing: `answer.lower() == "yes"`

✗ Forgetting () on methods

`name.upper` does nothing – it is the method itself, not the result. Always add `()`: `name.upper()`

✗ Wrong slice index

`name[1:3]` gives characters at index 1 and 2 – index 3 is NOT included. To get the first 3 letters:

`name[0:3]` or just `name[:3]`

✗ Methods don't change the original

`name.upper()` returns a new string. To keep the change: `name = name.upper()`

✗ Using int() without checking first

`int("hello")` crashes. Use `.isdigit()` first when the value comes from `input()`.

Part 7 – Mini Project



Mini Project – Name Formatter

Ask the user for their **first name** and **last name** separately. Then print:

1. Full name in title case: Sara Ahmed
2. Full name in uppercase: SARA AHMED
3. Formatted as last name first: AHMED, Sara
4. Initials: S.A.
5. How many characters in the full name (no spaces)
6. Full name reversed



20 minutes

What we learned today

- **Indexing:** `name[0]` , `name[-1]` – access individual characters
- **Slicing:** `name[0:3]` , `name[:: -1]` – extract parts of a string
- **Case methods:** `.upper()` `.lower()` `.title()` – and why `.lower()` matters for comparisons
- **Cleaning:** `.strip()` removes unwanted spaces · `.replace()` swaps text
- **Escape characters:** `\n` new line · `\t` tab
- **Checking:** `.isdigit()` `.isalpha()` `.startswith()` `.endswith()` · `in`
- **Searching:** `.count()` `.find()`
- **Splitting:** `.split()` breaks text into a list · `.join()` puts it back
- **f-strings:** `f"Hello, {name}!"` – cleaner than joining with `+`

Homework

Beginner

1. **Case practice** – create a variable with your full name in mixed case (e.g. `"sArA aHmEd"`). Print it in uppercase, lowercase, and title case.
2. **Strip and clean** – create `text = " Hello World! "` . Strip the spaces, replace "World" with your city, and print the length of the result.
3. **Count letters** – ask the user to enter a sentence. Print how many times the letter `"a"` appears (case-insensitive – use `.lower()` first).
4. **Word counter** – ask the user to type a sentence. Split it and print how many words it has.
5. **First and last** – ask the user for their full name. Print the first character and the last character.

Intermediate

6. **Name formatter** – ask for first name and last name. Print them as `"LAST, First"` (last name uppercase, first name title case).
7. **Word lengths** – ask the user for a sentence. Split it into words and print each word followed by its length:



Next Week – Conditions and Logic

Next lesson we will look at:

- `if`, `elif`, `else` – making decisions in code
- Comparison operators: `=`, `≠`, `>`, `<`, `≥`, `≤`
- Logical operators: `and`, `or`, `not`
- Combining conditions

Great work today!



See you next week.